

Aula 23 - Composables e Composition API Avançada

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Revisão da Aula Anterior (15 min)

- Discussão sobre:
 - Vue Router e navegação programática.
 - Gerenciamento de estado com Pinia.

2. Composition API vs Options API (20 min)

Feature	Composition API	Options API
Organização	Por função (reatividade, lifecycle)	Por opções (<code>data</code> , <code>methods</code>)
Reutilização	Composables (<code>useXxx</code>)	Mixins
TypeScript	Melhor suporte	Suporte limitado

3. Composables (25 min)

O Que São?

- Funções que encapsulam lógica reutilizável usando Composition API.
- Convenção de nome: `useNomeDoRecurso` .

Exemplo Básico:

javascript

```
// composables/useCounter.js
import { ref } from 'vue'

export function useCounter(initialValue = 0) {
  const count = ref(initialValue)

  const increment = () => count.value++
  const decrement = () => count.value--
```

```
    return { count, increment, decrement }  
  }  
}
```

Parte Prática (3 horas)

Atividade: Construindo Composables Customizados

Passo 1: useFetch (1h)

1. Implementação (composables/useFetch.js):

javascript

```
import { ref } from 'vue'  
  
export function useFetch(url) {  
  const data = ref(null)  
  const error = ref(null)  
  const loading = ref(false)  
  
  const fetchData = async () => {  
    loading.value = true  
    try {  
      const response = await fetch(url)  
      data.value = await response.json()  
    } catch (err) {  
      error.value = err  
    } finally {  
      loading.value = false  
    }  
  }  
  
  return { data, error, loading, fetchData }  
}
```

2. Uso em um Componente:

javascript

```
<script setup>  
import { useFetch } from '@composables/useFetch'  
  
const { data: posts, loading, fetchData } = useFetch('https://jsonplaceholder.typicode.com/posts')
```

```
fetchData() // Chamada inicial
</script>
```

Passo 2: useLocalStorage (1h)

1. Implementação (composables/useLocalStorage.js):

```
javascript

import { ref, watchEffect } from 'vue'

export function useLocalStorage(key, defaultValue) {
  const storedValue = localStorage.getItem(key)
  const value = ref(storedValue ? JSON.parse(storedValue) : defaultValue)

  watchEffect(() => {
    localStorage.setItem(key, JSON.stringify(value.value))
  })

  return value
}
```

2. Uso em um Componente:

```
javascript

<script setup>
import { useLocalStorage } from '@/composables/useLocalStorage'

const darkMode = useLocalStorage('darkMode', false)
</script>
```

Passo 3: useMouseTracker (1h)

1. Implementação (composables/useMouseTracker.js):

```
javascript

import { ref, onMounted, onUnmounted } from 'vue'

export function useMouseTracker() {
  const x = ref(0)
  const y = ref(0)

  const update = (e) => {
    x.value = e.clientX
    y.value = e.clientY
  }

  onMounted(() => window.addEventListener('mousemove', update))
```

```
onUnmounted(() => window.removeEventListener('mousemove', update))

return { x, y }
}
```

2. Uso em um Componente:

```
html

<template>
  <div>Mouse position: {{ x }}, {{ y }}</div>
</template>

<script setup>
import { useMouseTracker } from '@composables/useMouseTracker'
const { x, y } = useMouseTracker()
</script>
```

Tarefa para Casa

1. Criar um `useDebounce` :

- Implementar um composable que atrase a execução de funções (útil para busca em tempo real).

2. Integrar com Pinia:

- Modificar `useFetch` para usar um store Pinia como cache.

Próxima Aula

- **Tópico:** Testes em Vue (Vitest + Testing Library).
- **Atividade prática:** Testar componentes e composables.

Resultado Esperado

- 3 composables reutilizáveis:
 - `useFetch` : Gerencia chamadas HTTP.
 - `useLocalStorage` : Persistência de estado.
 - `useMouseTracker` : Rastreamento de cursor.

plaintext

- ✓ Lógica encapsulada em composables
- ✓ Reutilização entre componentes
- ✓ Pronto para testes automatizados

<https://vuejs.org/assets/composition-api-after.e3f2c350.png> (*Fluxo da Composition API*) 🚀